



Linux Command Handbook

Essential Linux Commands for Software Engineers • DevOps • Cloud • Software Architects



Chunk 1 (Commands 1–50)

Linux Fundamentals



Category 1: Navigation & Paths (Commands 1–8)

#	Command	Short Definition (6–10 words)	Example
1	<code>pwd</code>	Displays absolute path of current working directory.	<code>pwd</code>
2	<code>ls</code>	Lists files and directories in current location.	<code>ls</code>
3	<code>ls -l</code>	Lists files with detailed information and permissions.	<code>ls -l</code>
4	<code>ls -la</code>	Lists all files including hidden ones.	<code>ls -la</code>
5	<code>cd</code>	Changes current directory to specified destination path.	<code>cd /home/user</code>
6	<code>cd ..</code>	Moves one directory level above current location.	<code>cd ..</code>
7	<code>cd -</code>	Switches directly to current user's home directory.	<code>cd -</code>
8	<code>tree</code>	Displays directory structure in hierarchical tree format.	<code>tree</code>



Category 2: Files & Directories (Commands 9–22)

#	Command	Short Definition (6–10 words)	Example
9	<code>mkdir</code>	Creates one or multiple new directories.	<code>mkdir project</code>

#	Command	Short Definition (6–10 words)	Example
10	<code>mkdir -p</code>	Creates nested directories without intermediate errors.	<code>mkdir -p app/src</code>
11	<code>rmdir</code>	Removes empty directory from the filesystem safely.	<code>rmdir temp</code>
12	<code>touch</code>	Creates empty file or updates file timestamp.	<code>touch index.html</code>
13	<code>cp</code>	Copies files or directories to another location.	<code>cp file.txt backup/</code>
14	<code>cp -r</code>	Copies directories recursively with all contents included.	<code>cp -r app backup</code>
15	<code>mv</code>	Moves or renames files and directories efficiently.	<code>mv old.txt new.txt</code>
16	<code>rm</code>	Permanently removes files from the filesystem.	<code>rm notes.txt</code>
17	<code>rm -r</code>	Recursively removes directories and their contents.	<code>rm -r temp</code>
18	<code>rm -rf</code>	Forcefully removes directories without confirmation prompts.	<code>rm -rf build</code>
19	<code>ln</code>	Creates hard links between filesystem objects.	<code>ln file.txt link.txt</code>
20	<code>ln -s</code>	Creates symbolic links pointing to target files.	<code>ln -s app shortcut</code>
21	<code>stat</code>	Displays detailed metadata about specified filesystem objects.	<code>stat app.log</code>
22	<code>file</code>	Identifies actual file type using content analysis.	<code>file image.png</code>

Category 3: Viewing Files (Commands 23–30)

#	Command	Short Definition (6–10 words)	Example
23	<code>cat</code>	Displays complete contents of specified text files.	<code>cat notes.txt</code>
24	<code>less</code>	Views large files interactively one page at time.	<code>less app.log</code>
25	<code>more</code>	Displays file contents one screen at time.	<code>more data.txt</code>
26	<code>head</code>	Displays first ten lines of specified file.	<code>head app.log</code>
27	<code>head -n</code>	Displays specified number of initial file lines.	<code>head -20 app.log</code>
28	<code>tail</code>	Displays last ten lines of specified file.	<code>tail app.log</code>
29	<code>tail -f</code>	Continuously monitors appended data in log files.	<code>tail -f nginx.log</code>
30	<code>nl</code>	Displays file contents with line numbers included.	<code>nl notes.txt</code>

Category 4: Searching & Text Processing (Commands 31–45)

#	Command	Short Definition (6–10 words)	Example
31	<code>find</code>	Searches files and directories using specified search criteria.	<code>find . -name "*.log"</code>
32	<code>locate</code>	Quickly finds files using indexed system database.	<code>locate nginx.conf</code>
33	<code>grep</code>	Searches matching text patterns within files efficiently.	<code>grep "error" app.log</code>
34	<code>grep -r</code>	Recursively searches matching text inside directories.	<code>grep -r "TODO" src/</code>
35	<code>awk</code>	Processes structured text using pattern-action programming.	<code>awk '{print \$1}' file.txt</code>

#	Command	Short Definition (6–10 words)	Example
36	<code>sed</code>	Edits and transforms text streams automatically.	<code>sed 's/old/new/g' file.txt</code>
37	<code>sort</code>	Sorts text lines alphabetically or numerically ascending.	<code>sort names.txt</code>
38	<code>uniq</code>	Removes adjacent duplicate lines from sorted output.	<code>sort file.txt uniq</code>
39	<code>cut</code>	Extracts selected columns or character ranges efficiently.	<code>cut -d: -f1 /etc/passwd</code>
40	<code>tr</code>	Translates or deletes specified input characters.	<code>echo "abc" tr a-z A-Z</code>
41	<code>wc</code>	Counts lines, words, characters, and bytes accurately.	<code>wc -l app.log</code>
42	<code>tee</code>	Writes command output to file and terminal.	<code>ls tee files.txt</code>
43	<code>xargs</code>	Builds command arguments from standard input stream.	<code>find . -name "*.log" xargs rm</code>
44	<code>diff</code>	Compares two files and shows line differences.	<code>diff old.txt new.txt</code>
45	<code>paste</code>	Merges corresponding lines from multiple input files.	<code>paste file1.txt file2.txt</code>

Category 5: Help & Documentation (Commands 46–50)

#	Command	Short Definition (6–10 words)	Example
46	<code>man</code>	Displays complete manual page for specified command.	<code>man grep</code>
47	<code>--help</code>	Shows quick command usage and available options.	<code>ls --help</code>
48	<code>which</code>	Displays executable location found in system PATH.	<code>which python3</code>

#	Command	Short Definition (6–10 words)	Example
49	<code>whereis</code>	Locates binary, source, and manual page files.	<code>whereis ssh</code>
50	<code>history</code>	Displays previously executed terminal commands chronologically.	<code>history</code>

✓ Chunk 1 Complete (Commands 1–50)

Covered Categories:

- 📁 Navigation & Paths (1–8)
- 📁 Files & Directories (9–22)
- 📄 Viewing Files (23–30)
- 🔍 Searching & Text Processing (31–45)
- 📖 Help & Documentation (46–50)

The next chunk will continue with the same formatting, starting from Command #51 under **Linux Administration**.

📦 Chunk 2 (Commands 51–100)

Linux Administration

🔒 Category 6: File Permissions & Ownership (Commands 51–60)

#	Command	Short Definition (6–10 words)	Example
51	<code>chmod</code>	Changes file or directory access permissions securely.	<code>chmod 755 script.sh</code>
52	<code>chmod +x</code>	Grants executable permission to specified file.	<code>chmod +x deploy.sh</code>
53	<code>chmod -R</code>	Recursively changes permissions for entire directory.	<code>chmod -R 755 project/</code>
54	<code>chown</code>	Changes file or directory ownership to user.	<code>chown ubuntu app.log</code>

#	Command	Short Definition (6–10 words)	Example
55	<code>chown -R</code>	Recursively changes ownership of directory contents.	<code>chown -R nginx:nginx /var/www</code>
56	<code>chgrp</code>	Changes group ownership of specified files.	<code>chgrp developers app.log</code>
57	<code>umask</code>	Sets default permissions for newly created files.	<code>umask 022</code>
58	<code>getfacl</code>	Displays Access Control Lists for files.	<code>getfacl app.log</code>
59	<code>setfacl</code>	Configures Access Control Lists on files.	<code>setfacl -m u:john:rwX file.txt</code>
60	<code>ls -l</code>	Displays permissions, ownership, and detailed file information.	<code>ls -l</code>

Category 7: User & Group Management (Commands 61–75)

#	Command	Short Definition (6–10 words)	Example
61	<code>whoami</code>	Displays username of currently logged-in user.	<code>whoami</code>
62	<code>id</code>	Displays user ID and group memberships.	<code>id</code>
63	<code>who</code>	Lists users currently logged into system.	<code>who</code>
64	<code>groups</code>	Displays groups associated with current user.	<code>groups</code>
65	<code>useradd</code>	Creates a new user account securely.	<code>sudo useradd alok</code>
66	<code>adduser</code>	Creates user account with interactive prompts.	<code>sudo adduser alok</code>
67	<code>passwd</code>	Sets or changes user account password.	<code>passwd alok</code>
68	<code>usermod</code>	Modifies existing user account properties safely.	<code>usermod -aG sudo alok</code>
69	<code>userdel</code>	Deletes user account from the system.	<code>userdel alok</code>
70	<code>groupadd</code>	Creates a new user group.	<code>groupadd developers</code>

#	Command	Short Definition (6–10 words)	Example
71	<code>groupmod</code>	Modifies existing user group properties.	<code>groupmod developers</code>
72	<code>groupdel</code>	Deletes an existing user group.	<code>groupdel developers</code>
73	<code>su</code>	Switches current session to another user.	<code>su root</code>
74	<code>sudo</code>	Executes commands with administrative privileges safely.	<code>sudo apt update</code>
75	<code>visudo</code>	Safely edits sudoers configuration without syntax errors.	<code>sudo visudo</code>

Category 8: Process Management (Commands 76–90)

#	Command	Short Definition (6–10 words)	Example
76	<code>ps</code>	Displays currently running processes with details.	<code>ps</code>
77	<code>ps -ef</code>	Displays all running system processes comprehensively.	<code>ps -ef</code>
78	<code>top</code>	Monitors running processes and resource usage live.	<code>top</code>
79	<code>htop</code>	Provides interactive real-time process management interface.	<code>htop</code>
80	<code>kill</code>	Terminates process using specified process identifier.	<code>kill 1234</code>
81	<code>kill -9</code>	Forcefully terminates unresponsive running process immediately.	<code>kill -9 1234</code>
82	<code>pkill</code>	Terminates processes matching specified process name.	<code>pkill nginx</code>
83	<code>killall</code>	Terminates all processes with matching name.	<code>killall chrome</code>
84	<code>jobs</code>	Lists background jobs in current shell session.	<code>jobs</code>

#	Command	Short Definition (6–10 words)	Example
85	<code>bg</code>	Resumes stopped job execution in background.	<code>bg %1</code>
86	<code>fg</code>	Brings background job into foreground execution.	<code>fg %1</code>
87	<code>nohup</code>	Keeps process running after terminal logout.	<code>nohup java -jar app.jar &</code>
88	<code>nice</code>	Starts process with adjusted CPU priority.	<code>nice -10 script.sh</code>
89	<code>renice</code>	Changes CPU priority of running processes.	<code>renice 5 1234</code>
90	<code>pidof</code>	Displays process identifier for specified program.	<code>pidof nginx</code>



Category 9: System Information (Commands 91–100)

#	Command	Short Definition (6–10 words)	Example
91	<code>uname</code>	Displays basic operating system kernel information.	<code>uname -a</code>
92	<code>hostname</code>	Displays current system hostname or network name.	<code>hostname</code>
93	<code>hostnamectl</code>	Displays or modifies system hostname configuration.	<code>hostnamectl</code>
94	<code>uptime</code>	Shows system running duration and load averages.	<code>uptime</code>
95	<code>date</code>	Displays or sets current system date.	<code>date</code>
96	<code>cal</code>	Displays calendar for specified month or year.	<code>cal</code>
97	<code>free</code>	Displays system memory and swap usage.	<code>free -h</code>
98	<code>lscpu</code>	Displays detailed processor architecture and information.	<code>lscpu</code>

#	Command	Short Definition (6–10 words)	Example
99	<code>lsmem</code>	Displays installed physical memory layout information.	<code>lsmem</code>
100	<code>env</code>	Displays all current environment variables available.	<code>env</code>

✓ Chunk 2 Complete (Commands 51–100)

Covered Categories:

-  File Permissions & Ownership (51–60)
-  User & Group Management (61–75)
-  Process Management (76–90)
-  System Information (91–100)



Chunk 3 (Commands 101–150)

Server Administration



Category 10: Disk & Storage Management (Commands 101–112)

#	Command	Short Definition (6–10 words)	Example
101	<code>df</code>	Displays filesystem disk space usage statistics.	<code>df -h</code>
102	<code>du</code>	Displays disk usage for files and directories.	<code>du -sh project/</code>
103	<code>lsblk</code>	Lists available block storage devices and partitions.	<code>lsblk</code>
104	<code>blkid</code>	Displays filesystem UUIDs and partition information.	<code>sudo blkid</code>
105	<code>fdisk</code>	Creates, deletes, and manages disk partitions.	<code>sudo fdisk /dev/sda</code>
106	<code>mount</code>	Mounts filesystem to specified directory location.	<code>sudo mount /dev/sdb1 /mnt</code>

#	Command	Short Definition (6–10 words)	Example
107	<code>umount</code>	Safely unmounts mounted filesystem from directory.	<code>sudo umount /mnt</code>
108	<code>mkfs</code>	Creates filesystem on specified storage partition.	<code>sudo mkfs.ext4 /dev/sdb1</code>
109	<code>fsck</code>	Checks and repairs filesystem consistency errors.	<code>sudo fsck /dev/sdb1</code>
110	<code>sync</code>	Flushes filesystem buffers to physical storage safely.	<code>sync</code>
111	<code>dd</code>	Copies and converts files or disk images.	<code>dd if=/dev/sda of=backup.img</code>
112	<code>mountpoint</code>	Checks whether directory is mounted filesystem.	<code>mountpoint /mnt</code>

Category 11: Networking (Commands 113–132)

#	Command	Short Definition (6–10 words)	Example
113	<code>ip</code>	Configures and displays network interface information.	<code>ip addr</code>
114	<code>ping</code>	Tests network connectivity with remote host.	<code>ping google.com</code>
115	<code>traceroute</code>	Displays network path packets travel through.	<code>traceroute google.com</code>
116	<code>tracepath</code>	Traces packet route without administrative privileges.	<code>tracepath google.com</code>
117	<code>ss</code>	Displays active network sockets and connections.	<code>ss -tuln</code>
118	<code>netstat</code>	Displays network connections, routes, and statistics.	<code>netstat -tuln</code>
119	<code>curl</code>	Transfers data from or to remote servers.	<code>curl https://example.com</code>
120	<code>wget</code>	Downloads files directly from internet servers.	<code>wget https://example.com/file.zip</code>

#	Command	Short Definition (6–10 words)	Example
121	scp	Securely copies files between remote systems.	scp file.txt user@host:/home/user
122	rsync	Efficiently synchronizes files between local or remote systems.	rsync -av src/ backup/
123	ssh	Securely connects to remote Linux servers.	ssh user@server
124	sftp	Securely transfers files using SSH protocol.	sftp user@server
125	dig	Queries DNS servers for domain information.	dig google.com
126	nslookup	Looks up domain names and IP addresses.	nslookup google.com
127	host	Resolves hostnames into corresponding IP addresses.	host google.com
128	hostname -I	Displays all assigned local IP addresses.	hostname -I
129	arp	Displays and manages Address Resolution Protocol cache.	arp -a
130	route	Displays or modifies kernel routing table.	route -n
131	nc	Tests network ports and transfers raw data.	nc -zv localhost 80
132	telnet	Connects to remote services for connectivity testing.	telnet localhost 80

Category 12: Package Management (Commands 133–140)

#	Command	Short Definition (6–10 words)	Example
133	apt	Installs, removes, and manages Debian packages.	sudo apt install nginx
134	apt update	Updates package repository metadata from servers.	sudo apt update

#	Command	Short Definition (6–10 words)	Example
135	<code>apt upgrade</code>	Upgrades installed packages to latest versions.	<code>sudo apt upgrade</code>
136	<code>apt remove</code>	Removes installed package while preserving configuration.	<code>sudo apt remove nginx</code>
137	<code>apt purge</code>	Completely removes package including configuration files.	<code>sudo apt purge nginx</code>
138	<code>dpkg</code>	Installs or manages local Debian package files.	<code>sudo dpkg -i app.deb</code>
139	<code>yum</code>	Manages packages on RHEL and CentOS systems.	<code>sudo yum install httpd</code>
140	<code>dnf</code>	Modern package manager replacing yum on Fedora.	<code>sudo dnf install nginx</code>

Category 13: Services & Log Management (Commands 141–150)

#	Command	Short Definition (6–10 words)	Example
141	<code>systemctl</code>	Manages system services and startup behavior.	<code>systemctl status nginx</code>
142	<code>systemctl start</code>	Starts specified service immediately on system.	<code>sudo systemctl start nginx</code>
143	<code>systemctl stop</code>	Stops specified running service safely.	<code>sudo systemctl stop nginx</code>
144	<code>systemctl restart</code>	Restarts specified service to apply changes.	<code>sudo systemctl restart nginx</code>
145	<code>systemctl enable</code>	Enables service to start during boot.	<code>sudo systemctl enable nginx</code>
146	<code>systemctl disable</code>	Prevents service from starting during boot.	<code>sudo systemctl disable nginx</code>
147	<code>journalctl</code>	Displays logs collected by systemd journal service.	<code>journalctl -u nginx</code>
148	<code>dmesg</code>	Displays Linux kernel boot and hardware messages.	<code>dmesg</code>

#	Command	Short Definition (6–10 words)	Example
149	<code>watch</code>	Repeatedly executes command and refreshes output.	<code>watch df -h</code>
150	<code>lsof</code>	Lists files currently opened by running processes.	<code>lsof -i :80</code>

✓ Chunk 3 Complete (Commands 101–150)

Covered Categories:

-  Disk & Storage Management (101–112)
-  Networking (113–132)
-  Package Management (133–140)
-  Services & Log Management (141–150)

Chunk 4A (Commands 151–175)

DevOps Foundations

Category 14: Scheduling & Automation (Commands 151–162)

#	Command	Short Definition (6–10 words)	Example
151	<code>crontab</code>	Manages scheduled tasks for current user account.	<code>crontab -e</code>
152	<code>crontab -e</code>	Opens current user's cron table for editing.	<code>crontab -e</code>
153	<code>crontab -l</code>	Lists all scheduled cron jobs for user.	<code>crontab -l</code>
154	<code>crontab -r</code>	Removes all scheduled cron jobs permanently.	<code>crontab -r</code>
155	<code>cron</code>	Executes scheduled background tasks automatically every minute.	<code>sudo systemctl status cron</code>
156	<code>at</code>	Schedules one-time command execution at specified time.	<code>at 10:30 PM</code>

#	Command	Short Definition (6–10 words)	Example
157	<code>atq</code>	Lists all pending one-time scheduled jobs.	<code>atq</code>
158	<code>atrm</code>	Removes specified pending scheduled job safely.	<code>atrm 2</code>
159	<code>batch</code>	Executes commands automatically when system load decreases.	<code>batch</code>
160	<code>sleep</code>	Pauses command execution for specified duration.	<code>sleep 10</code>
161	<code>time</code>	Measures execution duration of specified command accurately.	<code>time ls -la</code>
162	<code>timeout</code>	Stops command after specified execution time expires.	<code>timeout 30 ping google.com</code>



Category 15: Shell & Environment (Commands 163–175)

#	Command	Short Definition (6–10 words)	Example
163	<code>echo</code>	Prints text or variable values to terminal.	<code>echo "Hello Linux"</code>
164	<code>printf</code>	Prints formatted output using specified format string.	<code>printf "%s\n" "Hello"</code>
165	<code>export</code>	Sets environment variables for current shell session.	<code>export JAVA_HOME=/usr/lib/jvm/java-21</code>
166	<code>unset</code>	Removes specified environment variable from current session.	<code>unset JAVA_HOME</code>
167	<code>env</code>	Displays or runs commands with custom environment.	<code>env</code>
168	<code>printenv</code>	Displays current environment variables and assigned values.	<code>printenv PATH</code>
169	<code>alias</code>	Creates shortcut for frequently used shell commands.	<code>alias ll='ls -la'</code>
170	<code>unalias</code>	Removes previously created shell command alias.	<code>unalias ll</code>

#	Command	Short Definition (6–10 words)	Example
171	<code>source</code>	Reloads shell configuration without restarting terminal.	<code>source ~/.bashrc</code>
172	<code>history</code>	Displays previously executed shell commands chronologically.	<code>history</code>
173	<code>clear</code>	Clears previous output from terminal screen completely.	<code>clear</code>
174	<code>reset</code>	Resets terminal settings and clears corrupted display.	<code>reset</code>
175	<code>exit</code>	Terminates current shell session or terminal window.	<code>exit</code>

✓ Chunk 4A Complete (Commands 151–175)

Covered Categories:

- 🕒 Category 14: Scheduling & Automation (151–162)
- 💻 Category 15: Shell & Environment (163–175)

Next: 📦 **Chunk 4B (Commands 176–205)** – Complete Git Essentials (30 Commands) covering configuration, branching, merging, rebasing, remotes, recovery, and collaboration.

📦 Chunk 4B (Commands 176–205)

Developer Essentials

🌳 Category 16: Git Essentials (Commands 176–205)

#	Command	Short Definition (6–10 words)	Example
176	<code>git config</code>	Configures Git username, email, and default settings.	<code>git config --global user.name "Alok"</code>
177	<code>git init</code>	Initializes new Git repository in current directory.	<code>git init</code>
178	<code>git clone</code>	Downloads remote repository into local machine.	<code>git clone https://github.com/user/repo.git</code>

#	Command	Short Definition (6–10 words)	Example
179	<code>git status</code>	Displays repository status and pending file changes.	<code>git status</code>
180	<code>git add</code>	Stages selected files for next commit.	<code>git add .</code>
181	<code>git restore</code>	Restores modified files to previous state.	<code>git restore app.js</code>
182	<code>git commit</code>	Saves staged changes into repository history permanently.	<code>git commit -m "Initial commit"</code>
183	<code>git log</code>	Displays complete commit history in chronological order.	<code>git log --oneline</code>
184	<code>git show</code>	Displays detailed information about specific commit.	<code>git show HEAD</code>
185	<code>git diff</code>	Shows differences between files, commits, or branches.	<code>git diff</code>
186	<code>git branch</code>	Lists, creates, or deletes Git branches.	<code>git branch</code>
187	<code>git switch</code>	Switches current working branch safely.	<code>git switch main</code>
188	<code>git checkout</code>	Switches branches or restores tracked files.	<code>git checkout feature-login</code>
189	<code>git merge</code>	Combines another branch into current branch.	<code>git merge feature-login</code>
190	<code>git rebase</code>	Reapplies commits onto another branch history.	<code>git rebase main</code>
191	<code>git stash</code>	Temporarily stores uncommitted working directory changes.	<code>git stash</code>
192	<code>git stash pop</code>	Restores latest stashed changes and removes stash.	<code>git stash pop</code>
193	<code>git fetch</code>	Downloads latest remote changes without merging.	<code>git fetch origin</code>
194	<code>git pull</code>	Fetches and merges latest remote repository changes.	<code>git pull origin main</code>
195	<code>git push</code>	Uploads local commits to remote repository.	<code>git push origin main</code>

#	Command	Short Definition (6–10 words)	Example
196	<code>git remote</code>	Manages remote repository connections and URLs.	<code>git remote -v</code>
197	<code>git tag</code>	Creates, lists, or deletes version tags.	<code>git tag v1.0.0</code>
198	<code>git reset</code>	Moves HEAD and optionally unstages changes.	<code>git reset HEAD~1</code>
199	<code>git revert</code>	Creates new commit reversing previous commit.	<code>git revert HEAD</code>
200	<code>git clean</code>	Removes untracked files and directories safely.	<code>git clean -fd</code>
201	<code>git cherry-pick</code>	Applies selected commit onto current branch.	<code>git cherry-pick a1b2c3d</code>
202	<code>git blame</code>	Shows line-by-line commit and author information.	<code>git blame app.js</code>
203	<code>git reflog</code>	Displays reference history for HEAD movements.	<code>git reflog</code>
204	<code>git bisect</code>	Finds commit introducing bug using binary search.	<code>git bisect start</code>
205	<code>git cherry</code>	Compares commits between local and upstream branches.	<code>git cherry -v</code>

✓ **Chunk 4B Complete (Commands 176–205)**

Covered Categories:

- 🌳 Category 16: Git Essentials (176–205)

This Git section now covers the complete day-to-day workflow expected of software engineers and DevOps professionals, including configuration, staging, commits, branching, merging, rebasing, remotes, recovery, debugging, and collaboration.

Next: 📦 **Chunk 4C (Commands 206–220)** – SSH & Security Essentials (15 Commands).



Chunk 4C (Commands 206–220)

Security & Remote Administration



Category 17: SSH & Security (Commands 206–220)

#	Command	Short Definition (6–10 words)	Example
206	ssh	Securely connects to remote Linux servers over SSH.	ssh ubuntu@192.168.1.10
207	ssh-keygen	Generates secure public and private SSH key pairs.	ssh-keygen -t ed25519
208	ssh-copy-id	Copies public SSH key to remote server.	ssh-copy-id ubuntu@server
209	scp	Securely copies files between local and remote systems.	scp app.jar user@server:/opt/apps
210	sftp	Securely transfers files using SSH file transfer.	sftp user@server
211	ssh-agent	Manages SSH private keys for authentication sessions.	eval \$(ssh-agent -s)
212	ssh-add	Adds private keys to running SSH agent.	ssh-add ~/.ssh/id_ed25519
213	openssl	Generates certificates and performs cryptographic operations securely.	openssl version
214	gpg	Encrypts, decrypts, and digitally signs sensitive files.	gpg -c secrets.txt
215	base64	Encodes or decodes data using Base64 encoding.	echo "Hello" base64
216	sha256sum	Calculates SHA-256 checksum for integrity verification.	sha256sum app.zip
217	sha512sum	Calculates SHA-512 checksum for stronger integrity verification.	sha512sum app.zip
218	md5sum	Calculates MD5 checksum for quick file comparison.	md5sum app.zip

#	Command	Short Definition (6–10 words)	Example
219	<code>hostnamectl</code>	Displays or changes system hostname configuration safely.	<code>hostnamectl set-hostname web-server</code>
220	<code>timedatectl</code>	Displays or configures system date and timezone.	<code>timedatectl status</code>

✓ Chunk 4C Complete (Commands 206–220)

Covered Categories:

- 🔒 Category 17: SSH & Security (206–220)

Notes

- `hostnamectl` and `timedatectl` are included because they're common administrative commands you'll use while provisioning Linux servers in cloud environments.
- Core SSH workflow is fully covered:
 - Remote login (`ssh`)
 - Key generation (`ssh-keygen`)
 - Key deployment (`ssh-copy-id`)
 - Secure copy (`scp`)
 - File transfer (`sftp`)
 - Key management (`ssh-agent` , `ssh-add`)
- Common cryptographic and integrity tools are included:
 - `openssl`
 - `gpg`
 - `base64`
 - `sha256sum`
 - `sha512sum`
 - `md5sum`

Next: 📦 **Chunk 4D (Commands 221–245)** – Docker Essentials (25 Commands), covering the complete day-to-day Docker workflow used in DevOps and cloud environments.

📦 Chunk 4D (Commands 221–245)

Docker Essentials

Note:

Docker isn't a built-in Linux command, but it's one of the most essential tools for modern Software Engineers, DevOps Engineers, Cloud Engineers, and Software Architects.

**Category 18: Docker Essentials (Commands 221–245)**

#	Command	Short Definition (6–10 words)	Example
221	<code>docker version</code>	Displays Docker client and server version information.	<code>docker version</code>
222	<code>docker info</code>	Displays detailed Docker engine configuration and status.	<code>docker info</code>
223	<code>docker login</code>	Authenticates with Docker registry using credentials.	<code>docker login</code>
224	<code>docker logout</code>	Removes stored Docker registry authentication credentials.	<code>docker logout</code>
225	<code>docker search</code>	Searches Docker Hub for available container images.	<code>docker search nginx</code>
226	<code>docker pull</code>	Downloads container image from Docker registry.	<code>docker pull nginx</code>
227	<code>docker images</code>	Lists locally available Docker container images.	<code>docker images</code>
228	<code>docker build</code>	Builds Docker image from Dockerfile instructions.	<code>docker build -t myapp .</code>
229	<code>docker tag</code>	Assigns new name or tag to image.	<code>docker tag myapp:latest myapp:v1</code>
230	<code>docker push</code>	Uploads Docker image to remote registry.	<code>docker push username/myapp:v1</code>
231	<code>docker run</code>	Creates and starts new Docker container.	<code>docker run -d nginx</code>
232	<code>docker ps</code>	Lists currently running Docker containers only.	<code>docker ps</code>
233	<code>docker ps -a</code>	Lists all Docker containers including stopped ones.	<code>docker ps -a</code>

#	Command	Short Definition (6–10 words)	Example
234	<code>docker exec</code>	Executes command inside running Docker container.	<code>docker exec -it web bash</code>
235	<code>docker logs</code>	Displays logs generated by Docker container.	<code>docker logs web</code>
236	<code>docker inspect</code>	Displays detailed metadata about Docker objects.	<code>docker inspect web</code>
237	<code>docker stop</code>	Stops running Docker container gracefully.	<code>docker stop web</code>
238	<code>docker start</code>	Starts previously stopped Docker container.	<code>docker start web</code>
239	<code>docker restart</code>	Restarts Docker container without manual intervention.	<code>docker restart web</code>
240	<code>docker rm</code>	Removes stopped Docker container permanently.	<code>docker rm web</code>
241	<code>docker rmi</code>	Removes unused Docker images from local storage.	<code>docker rmi nginx</code>
242	<code>docker network</code>	Creates, lists, and manages Docker networks.	<code>docker network ls</code>
243	<code>docker volume</code>	Creates and manages persistent Docker volumes.	<code>docker volume ls</code>
244	<code>docker compose</code>	Defines and runs multi-container Docker applications.	<code>docker compose up -d</code>
245	<code>docker system</code>	Manages Docker system resources and cleanup.	<code>docker system prune</code>

✓ **Chunk 4D Complete (Commands 221–245)**

Covered Topics:

- Docker installation verification
- Image management
- Registry operations
- Container lifecycle
- Container inspection
- Networks
- Volumes
- Docker Compose
- Docker cleanup

This set covers the core day-to-day Docker workflow used in professional development and DevOps. Advanced subcommands (such as `docker cp`, `docker stats`, `docker events`, `docker save`, `docker load`, and `docker image` / `docker container namespaces`) are useful but are used less frequently than the commands above.

Next: 📦 **Chunk 4E (Commands 246–265)** – Kubernetes & Cloud Utilities, covering `kubectl`, Helm, and JSON/YAML processing tools commonly used in cloud-native environments.

📦 **Chunk 4E (Commands 246–265)**

Kubernetes & Cloud Utilities

Note:

Kubernetes (`kubectl`), Helm, and utilities like `jq` / `yq` are not built into Linux, but they are essential tools for modern DevOps, Cloud, and Platform Engineers.

🔗 **Category 19: Kubernetes & Cloud Utilities (Commands 246–265)**

#	Command	Short Definition (6–10 words)	Example
246	<code>kubectl config</code>	Manages Kubernetes cluster contexts and credentials.	<code>kubectl config get-contexts</code>
247	<code>kubectl get</code>	Lists Kubernetes resources from selected namespace.	<code>kubectl get pods</code>

#	Command	Short Definition (6–10 words)	Example
248	<code>kubectl describe</code>	Displays detailed information about Kubernetes resources.	<code>kubectl describe pod nginx</code>
249	<code>kubectl apply</code>	Creates or updates resources from manifest files.	<code>kubectl apply -f deployment.yaml</code>
250	<code>kubectl delete</code>	Removes Kubernetes resources from current cluster.	<code>kubectl delete pod nginx</code>
251	<code>kubectl logs</code>	Displays logs generated by Kubernetes containers.	<code>kubectl logs nginx</code>
252	<code>kubectl exec</code>	Executes commands inside running Kubernetes containers.	<code>kubectl exec -it nginx -- bash</code>
253	<code>kubectl top</code>	Displays CPU and memory usage metrics.	<code>kubectl top pods</code>
254	<code>kubectl scale</code>	Changes replica count for Kubernetes workloads.	<code>kubectl scale deployment web --replicas=3</code>
255	<code>kubectl rollout</code>	Manages deployment rollouts and rollback operations.	<code>kubectl rollout status deployment/web</code>
256	<code>kubectl expose</code>	Creates Kubernetes Service for application access.	<code>kubectl expose deployment web --port=80</code>
257	<code>kubectl port-forward</code>	Forwards local ports to Kubernetes resources.	<code>kubectl port-forward pod/nginx 8080:80</code>
258	<code>kubectl edit</code>	Edits Kubernetes resources using default editor.	<code>kubectl edit deployment web</code>
259	<code>helm install</code>	Installs Kubernetes applications using Helm charts.	<code>helm install web nginx</code>
260	<code>helm upgrade</code>	Upgrades existing Helm release to newer version.	<code>helm upgrade web nginx</code>
261	<code>helm uninstall</code>	Removes Helm release from Kubernetes cluster.	<code>helm uninstall web</code>
262	<code>helm list</code>	Lists installed Helm releases in cluster.	<code>helm list</code>
263	<code>jq</code>	Processes and filters JSON data efficiently.	<code>kubectl get pods -o json jq '.items[].metadata.name'</code>

#	Command	Short Definition (6–10 words)	Example
264	<code>yq</code>	Processes and edits YAML configuration files.	<code>yq '.spec.replicas' deployment.yaml</code>
265	<code>watch</code>	Continuously monitors changing Kubernetes command outputs.	<code>watch kubectl get pods</code>

✓ ☒ Linux Command Handbook Complete (Commands 1–265)

Final Coverage

Chunk	Commands	Topics
Chunk 1	1–50	Navigation, Files, Viewing, Text Processing, Help
Chunk 2	51–100	Permissions, Users, Processes, System Information
Chunk 3	101–150	Storage, Networking, Packages, Services, Logs
Chunk 4A	151–175	Scheduling, Automation, Shell Environment
Chunk 4B	176–205	Complete Git Workflow
Chunk 4C	206–220	SSH, Security, Encryption, Checksums
Chunk 4D	221–245	Docker Essentials
Chunk 4E	246–265	Kubernetes, Helm, JSON/YAML Utilities

Skills Covered

This handbook prepares you for day-to-day work in:

- Software Engineering
- Full Stack Development
- DevOps Engineering
- Cloud Engineering (AWS, Azure, GCP)
- Site Reliability Engineering (SRE)
- Software Architecture
- Linux System Administration

Overall, this is a solid practical reference focused on the commands you'll encounter most often in development, deployment, automation, containerization, orchestration, and production troubleshooting.